Chapitre 5 :
Les Services web REST

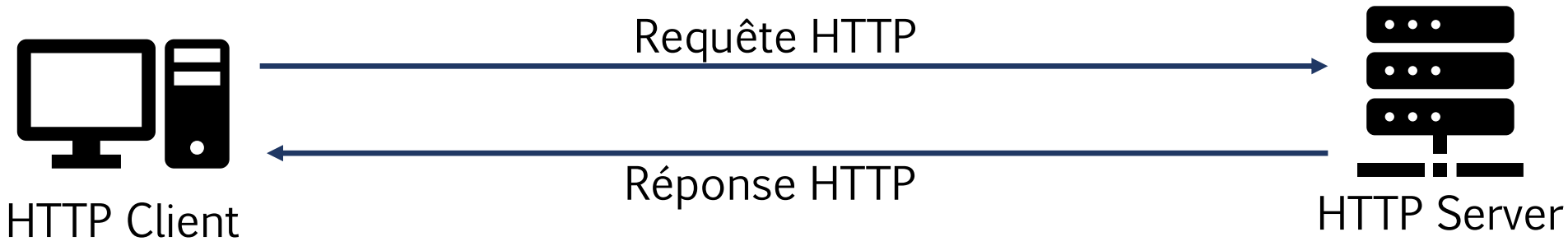


hamdi.aloulou@isima.u-monastir.tn

Rappel Protocole HTTP

Protocole HTTP

- HyperText Transfer Protocol
- Protocole d'échanges d'information sur le web
- Utilise les Requêtes / Réponses
- Basé sur TCP/IP



HTTP : Enchainement Client - Serveur

(1) L'utilisateur lance une URI à travers le navigateur http://host:port/path/file

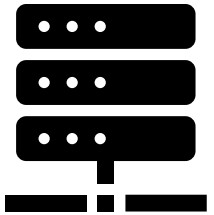


Client (Browser)

(2) Le navigateur envoie un message request

GET URI HTTP/1.1
URI:
http://host:port/path/file
……..

(3) Le serveur mappe l'URL vers un fichier ou le répertoire du document



Server (@host:port)

(4) Le serveur envoie un message response

HTTP/1.1 200 OK
……..
file
……..

(5) Le navigateur formatte la réponse et l'affiche

HTTP (over TCP/IP)

URI

- Unique Resource Identificator
- Identifie les ressources de manière unique sur le Web
- 4 parties
  - Protocole (http, ftp, mail, …)
  - Host (google.com)
  - Port (8080, 80)
  - Path (Chemin vers la ressource sur le serveur)

URI :

Protocole://Host:Port/Path

Méthodes HTTP

- HTTP définit un ensemble de méthode permettant de caractériser les requêtes
  - GET : Récupérer des ressources d'un serveur
  - POST : Envoyer des données à un serveur
  - PUT : Modifier des données
  - DELETE : Suppression de données
- Autres : HEAD, TRACE, CONNECT

Requêtes HTTP

- Permet à un client de demander une ressource sur un serveur
- Format d'un message HTTP
  - **Header (Entête)**
    - Request Line
    - Request Headers [Optional]
  - **Body (Corps)**

Requêtes HTTP

- Request Line

```
GET http://localhost:8080/bibliotheque/index.html HTTP/1.1
```

- Request Headers

```
Host: localhost:8080
Content-Length: 176
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Origin: http://localhost:8080
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_0) AppleWebKit/537.36 (KHTML,
likeGecko)
Content-Type: application/x-www-form-urlencoded
```

- Request Body

```
nom: licence
description: licence ISIMa
```

Réponse HTTP

- Réponse du serveur au client
- Format d'une réponse HTTP
  - **Response Header**
    - Response Line
    - Response Headers
  - **Response Body** [Optional]

Réponses HTTP

- Response Line

```
HTTP/1.1 200 OK
```

- Response Headers

```
Server: GlassFish Server Open Source Edition 4.0
Content-Type: text/html;charset=UTF-8
Date: Sun, 23 Nov 2014 16:05:39 GMT
Content-Length: 2274
```

- Response Body

```
<html xmlns="http://www.w3.org/1999/xhtml"><html xmlns="http://www.w3.org/1999/xhtml">
<head>
<link type="text/css" rel="stylesheet" href="/bibliotheque/faces/javax.faces.resource/theme.css?ln=primefaces-aristo"/>
<link type="text/css" rel="stylesheet" href="/bibliotheque/faces/javax.faces.resource/css/jsfcrud.css" />
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Create New Categorie</title>
</head>
<body>
<h1>Bienvenue dans cette licence</h1>
<div id="messagePanel"><table><tr style="color: green"><td>Ceci est une page statique pour décrire la licence.</td></tr>
</table>
</div>
</body>
</html>
```

Définition de Service Web REST

REST

Définition :

- Acronyme de **RE**presentational **S**tate **T**ransfert
- REST est un style d'architecture inspiré de l'architecture du web fortement basé sur le protocole HTTP
- Basée sur les méthodes HTTP (GET, POST, PUT, DELETE)
- Orienté Ressources, identifiées par des URI(s)

REST ➔ Caractéristiques

› Les services REST sont sans états

– Chaque requête envoyée au serveur doit contenir toutes les informations relatives à son état et est traitée indépendamment de toutes autres requêtes.

-> Minimisation des ressources systèmes (pas de gestion de session, ni d'état).

Invocation des APIs REST

REST : Manipulation de ressources

- Une ressource est un objet identifiable dans un système

  ➔ Livre, Personne, Client, Catégorie, Prêt

- Les **ressources** sont identifiées par des URIs. Une URI identifie de manière unique une ressource sur le système (https://jsonplaceholder.typicode.com/users/2)

Clef primaire de la ressource dans la BD

- Les types de requêtes HTTP (GET, POST, PUT, DELETE) vont permettre de **manipuler les ressources.**

- **Format d'échanges** entre le client et le serveur (XML, JSON, text/plain,…)

Action sur les ressources

- Une ressource peut subir quatre opérations de bases CRUD correspondant aux quatre principaux types de requêtes HTTP (GET, PUT, POST, DELETE)
- REST s'appuie sur le protocole HTTP pour effectuer ces opérations sur les objets
  - POST ➔ Créer une nouvelle ressource
  - GET ➔ Récupérer une ressource sans la modifier
  - PUT ➔ Mettre à jour une ressource identifié par l'URI
  - DELETE ➔ Supprimer la ressource identifiée par l'URI

Les étapes d'échange (1/3)

1. **Client – Génération d'un message**
   - Définition de la ressource demandée
   - Définition de l'action et donc de la méthode HTTP à utiliser (GET / POST / PUT / DELETE)

GET https://jsonplaceholder.typicode.com/users/2

Les étapes d'échange (2/3)

2. Serveur – Réception et Réponse

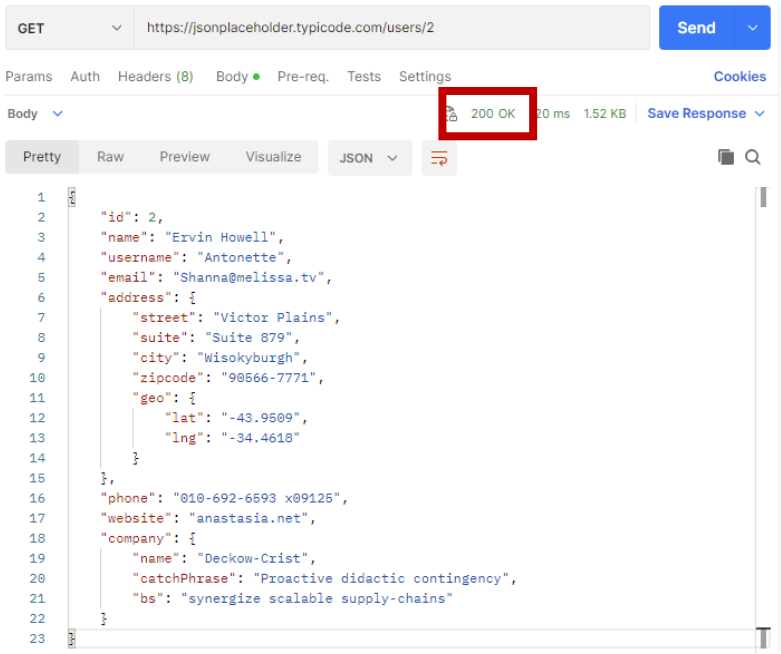
- Identification de la ressource demandée
- Identification de la méthode HTTP utilisée
- Résolution de l'action
- Génération de la réponse (Représentation de la ressource, Code d'état)



Les étapes d'échange (3/3)

3. Client – Lecture de la réponse

- Décodage du code d'état (code résultat, code erreur)
- Si positif (200/201) : validation (création, modification, suppression, consultation)
- Si négatif (4XX/5XX) : Information sur la raison de l'échec





Méthode POST

› La méthode POST crée une nouvelle ressource sur le système



Client

POST: https://jsonplaceholder.typicode.com/users/

Corps de la requête

```
{
    "name": "Mario"   ,
    "username": "Mario",
    "email": "Mario@karina.biz,
...
}
```



Serveur

Statut : 201, 204
Message : Created, No content
Corps de la réponse

```
{
"id": 11
}
```

Méthode GET

› La méthode GET renvoie une représentation de la ressource tel qu'elle est sur le système



Client

GET: https://jsonplaceholder.typicode.com/users/2

Statut : 200
Message : OK
Corps de la réponse


```
{
   "id": 2,
   "name": "Ervin Howell",
   "username": "Antonette",
   "email": "Shanna@melissa.tv",
   "address": {
      "street": "Victor Plains",
      "city": "Wisokyburgh",
      "zipcode": "90566-7771"
   },
   "phone": "010-692-6593 x09125«
}
```




Serveur

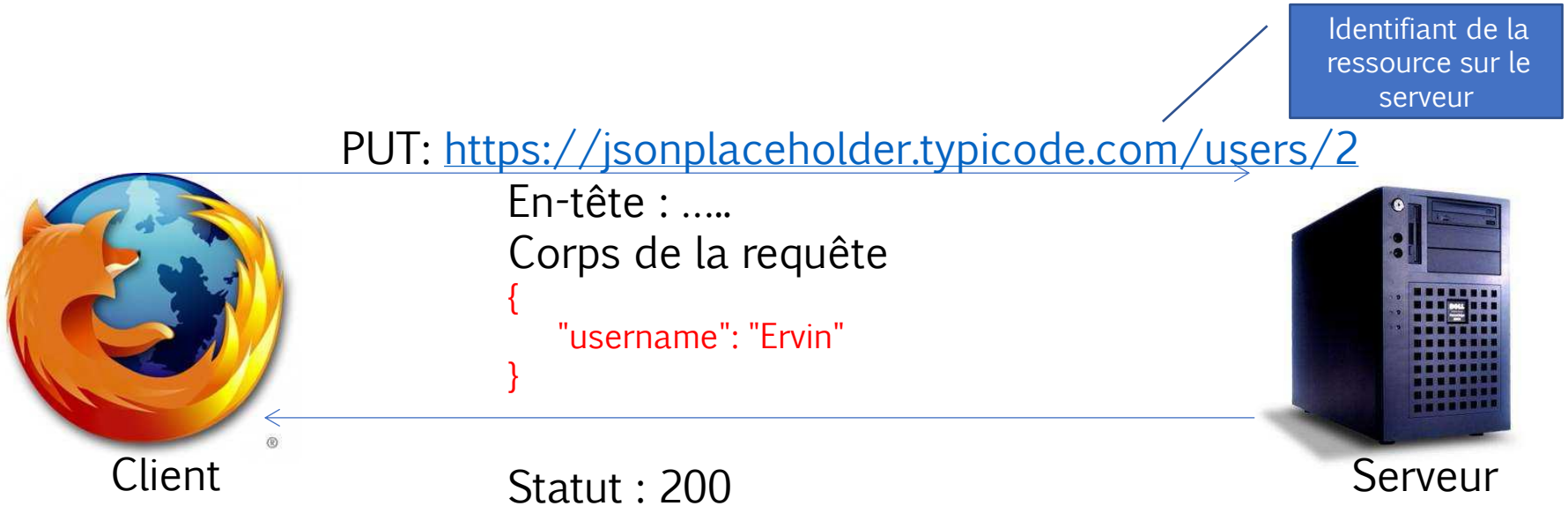
Méthode GET

› La méthode GET renvoie une représentation de la ressource tel qu'elle est sur le système



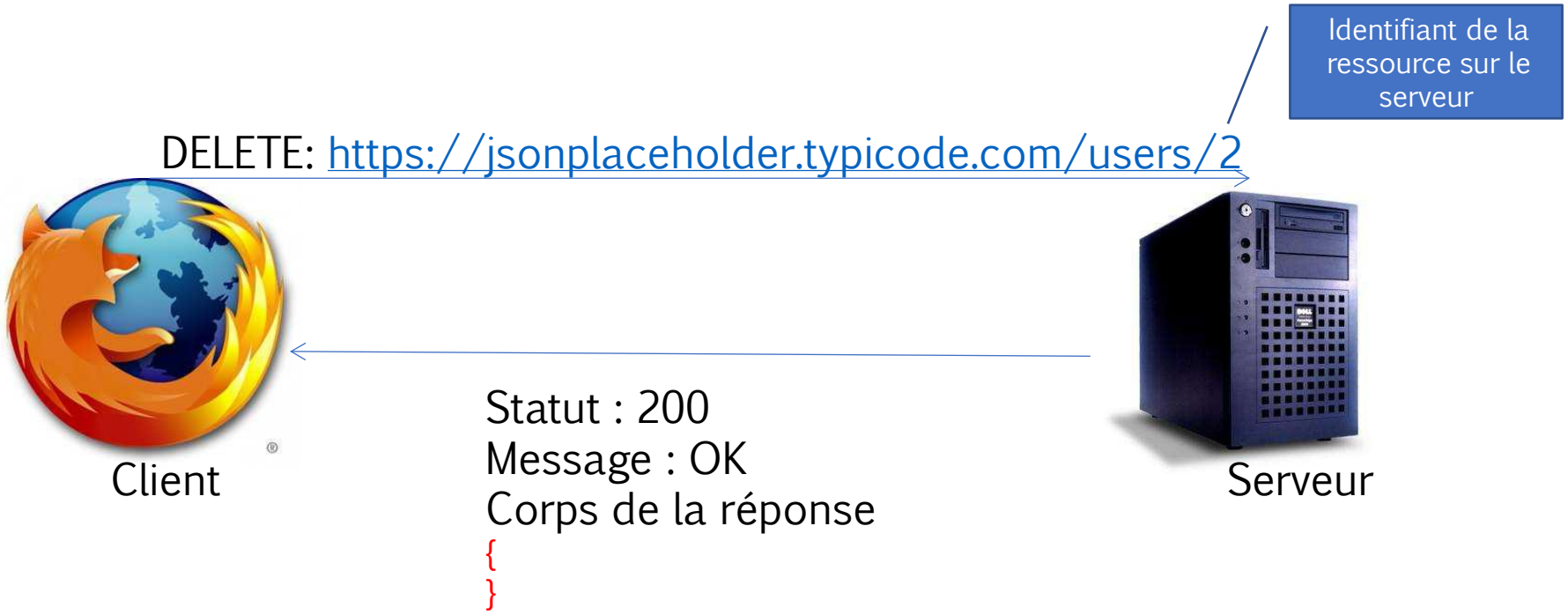
Méthode PUT

› Mise à jour de la ressource sur le système



Méthode DELETE

› Supprime la ressource identifiée par l'URI sur le serveur



REST, Méthode HTTP et Ressources

› L'opération à effectuer sur une ressource est déterminée par le type de la requête HTTP

› **Plusieurs actions sont possibles pour une même URI**

➔ **Tout dépend du type de la méthode HTTP utilisée**

Exemple :

GET
PUT
DELETE
→ https://jsonplaceholder.typicode.com/users/2

GET
POST
→ https://jsonplaceholder.typicode.com/users

Les services RESTFUL

› Que se passe-t-il

- si on fait de la lecture avec un POST ?
- Si on fait une mise à jour avec un DELETE ?
- Si on fait une suppression avec un PUT ?

➔ REST ne l'interdit pas

➔ Mais si vous le faites, votre application ne respecte pas les exigences REST et donc n'est pas RESTFUL

Représentation des données

- GET ➔ Le serveur renvoie au client l'état de la ressource
- PUT, POST ➔ Le client envoie l'état d'une ressource au serveur

Les données échangées entre le client et le serveur pour réaliser ces actions peuvent être sous différent format selon l'API REST développée / utilisée :

- JSON
- XML
- CSV
- Text/plain
- …

Le format de données JSON

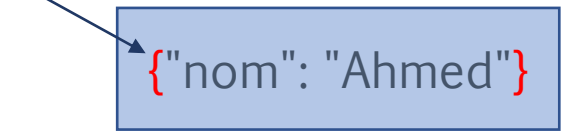
JSON

JSON « **J**ava**S**cript **O**bject **N**otation » est un format d'échange de données, facile à lire par un humain et interpréter par une machine.

Deux structures :

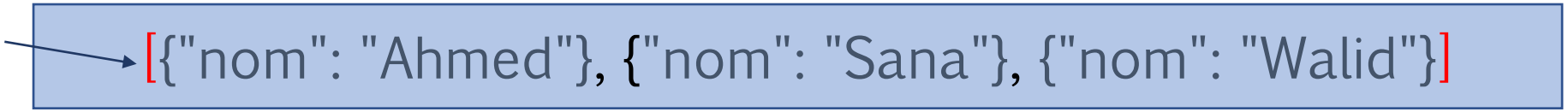
› Une collection de clefs/valeurs ➔ Object

Accolade



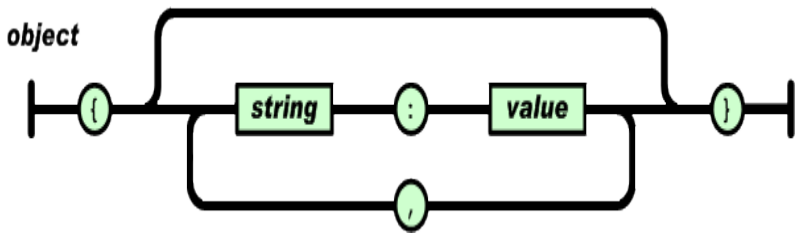
› Une collection ordonnée d'objets ➔ Array

Crochet



JSON Object

- Commence par une accolade « { » et se termine par une accolade « } »
- Composé d'une liste non ordonnée de paires clefs/valeurs
- Une clef est suivie de « : »
- Les paires clef/valeur sont séparés par « , »

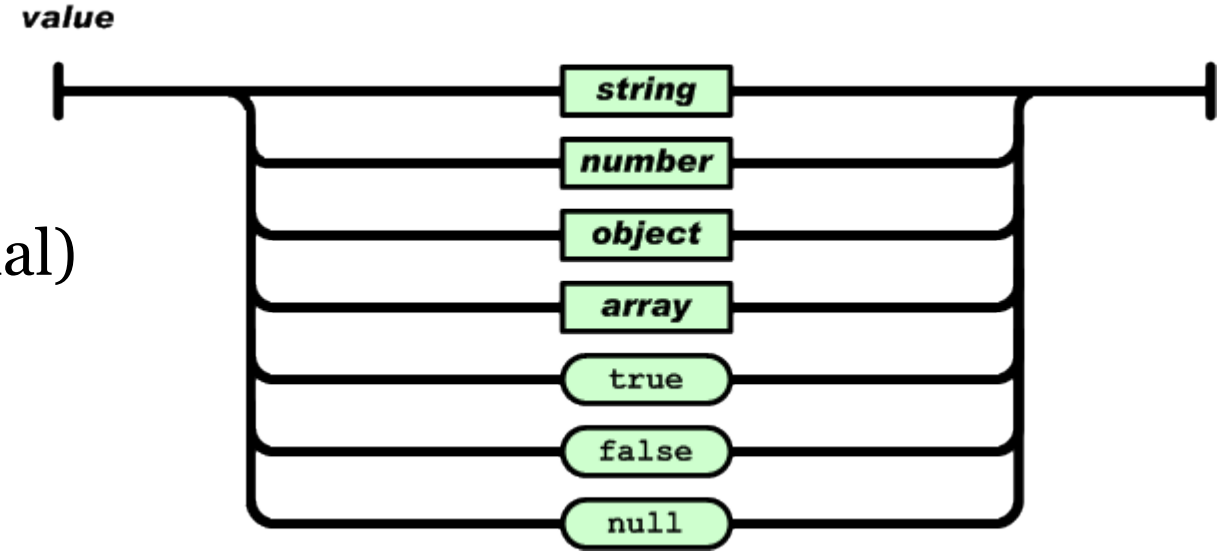



```
{ "id": 51,
"nom": "Mathematiques 1",
"resume": "Resume of math ",
"isbn": "123654",
"categorie":
      {
      "id": 2, "nom": "Mathematiques",
      "description": "Description of
      mathematiques "
      },
"quantite": 42,
"photo": ""
}
```

JSON Value

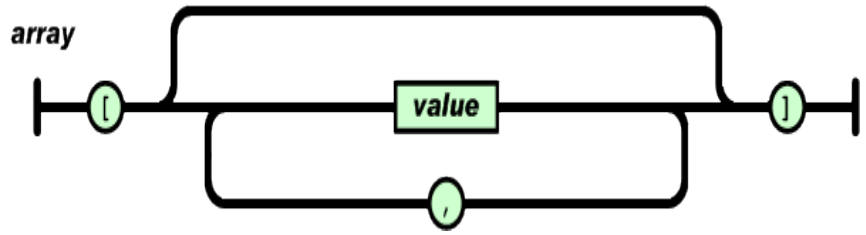
Une valeur peut être :

- Un string entre «""»
- Un nombre (entier, décimal)
- Un boolean (true, false)
- Un objet
- Null



JSON Array

- Liste ordonnée d'objets commençant par un croché « [« et se terminant par un croché «] »
- les objets dans un Array sont séparés l'un de l'autre par « , »




```
[
{ "id": 51,
"nom": "Mathematiques 1",
"resume": "Resume of math ",
"isbn": "123654",
"quantite": 42,
"photo": ""
},
{ "id": 102,
"nom": "Mathematiques 1",
"resume": "Resume of math ",
"isbn": "12365444455",
"quantite": 42,
"photo": ""
}
]
```

Exemple de document JSON

```
{
    "titre": "The Social network",
    "résumé": "On a fall night in 2003, Harvard undergrad and programming
              genius Mark Zuckerberg sits down at his computer and
              heatedly begins working on a new idea.(...)",
    "année": 2010,
    "réalisateur": {"nom": "Fincher", "prénom": "David"},
    "acteurs": [{"prénom": "Jesse", "nom": "Eisenberg"},
              {"prénom": "Rooney", "nom": "Mara"}]
}
```

Équivalant en XML

```
<film>
    <titre>The Social network</titre>
    <résumé>On a fall night in 2003, Harvard undergrad and programming genius Mark
     Zuckerberg sits down at his computer and heatedly begins working on a new idea.</résumé>
    <année>2010</année>
    <réalisateur>
        <nom>Fincher</nom>
        <prénom>David</prénom>
    </réalisateur>
    <acteur>
        <prénom>Jesse</prénom>
        <nom>Eisenberg</nom>
    </acteur>
    <acteur>
        <prénom>Rooney</prénom>
        <nom>Mara</nom>
    </acteur>
</film>
```

Pour s'exercer

- Comment savoir qu'un document JSON est bien formé (c'est-à-dire syntaxiquement correct)? Il existe des validateurs en ligne, bien utiles pour détecter les fautes.
- Essayez par exemple http://jsonlint.com/: copiez-collez les documents JSON donnés précédemment dans le validateur et vérifiez qu'ils sont corrects (ou pas…).
- Le document suivant contient (beaucoup) d'erreurs, à vous de les corriger. Cherchez-les visuellement, puis aidez-vous du validateur.

Pour s’exercer

```
{  "titre": "Taxi driver",
"année": 1976,
"genre": "drama",
"résumé": 'Vétéran de la Guerre du Vietnam,
Travis Bickle est chauffeur de taxi dans la ville
de New York.  La violence quotidienne l'affecte
peu à peu.’,
"pays": "USA",
"réalisateur": {
"nom": "Scorcese",
prénom: "Martin",
"date_naissance": "1962"
},
"acteurs": [
{nom: "Jodie",
"prénom": "Foster",
"date_naissance": null,
"rôle": "1962" }
{nom: "Robert",
"prénom": "De Niro",
"date_naissance": "1943",
"rôle": "Travis Bickle ", }
}
```

Pour s’exercer (Correction)


```
{  "titre": "Taxi driver",
"année": 1976,
"genre": "drama",
"résumé": "Vétéran de la Guerre du Vietnam,
Travis Bickle est chauffeur de taxi dans la ville
de New York.  La violence quotidienne l'affecte
peu à peu.",
"pays": "USA",
"réalisateur": {
"nom": "Scorcese",
"prénom": "Martin",
"date_naissance": "1962"
},
"acteurs": [
{"nom": "Jodie",
"prénom": "Foster",
"date_naissance": null,
"rôle": "1962" },
{"nom": "Robert",
"prénom": "De Niro",
"date_naissance": "1943",
"rôle": "Travis Bickle "} ]
}
```

Développement d'une API REST avec Spring Boot

Les annotations Spring boot

- @**SpringBootApplication** : annotation pour la classe main de l'application Spring boot. Elle Englobe les annotations :
  - **@Configuration** : donne à la classe actuelle la possibilité de définir des configurations qui iront remplacer les traditionnels fichiers XML. Ces configurations se font via des Beans.
  - **@EnableAutoConfiguration** : Permet, au démarrage de Spring, de générer automatiquement les configurations nécessaires en fonction des dépendances situées dans notre classpath.
  - **@ComponentScan** : indique qu'il faut scanner les classes de ce package afin de trouver des Beans de configuration.

Les annotations Spring boot pour les APIs REST

- **Annotations de classe :**
  - Placée au-dessus de la déclaration de la classe
  - **@RestController** : indique que la classe va pouvoir traiter des requêtes REST (GET, POST, PUT, DELETE)
  - **@RequestMapping** : pour définir le chemin racine de toutes les requêtes de ce contrôleur.
- **Annotations de méthode :**
  - Placées au-dessus des déclarations des méthodes
  - **@GetMapping, @PostMapping, @PutMapping, @DeleteMapping**
  - Peuvent accepter plusieurs paramètres :
    - value : indique l'URI à laquelle une méthode doit répondre.
    - produces/consumes : indique le format de données (XML/JSON) que la méthode est capable de répondre/consommer.

Exemples

```
@RestController
@RequestMapping("/api")
public class ProduitController {
    // toutes les méthodes ci-dessous auront le préfixe /api
}
```

› La classe **ProduitController** est déclarée comme un **API REST** à travers l'annotation **@RestController**

› Toutes **les méthodes** de cette classe seront **accessibles** en utilisant le préfixe **/api**

Exemples

```
@GetMapping("/produits")
 public List<Product> listeProduits() { … }
```

La méthode "listeProduits" sera invoquée suite à une requête HTTP qui utilise **la méthode HTTP "GET"** avec une URI qui pointe sur **la ressource "/api/produits"**.

```
@PostMapping("/produits")
 public void ajouterUnProduit(@RequestBody Product produit) { … }
```

La méthode "*ajouterUnProduit*" sera invoquée suite à une requête HTTP qui utilise **la méthode HTTP "POST"** avec une URI qui pointe sur **la ressource "/api/produits".**

Exemples

```
@GetMapping(value = "/produits" produces = "application/json")
 public List<Product> listeProduits() { … }
```

La méthode "listeProduits" sera invoquée suite à une requête HTTP qui utilise **la méthode HTTP "GET"** avec une URI qui pointe sur **la ressource "api/produits"**.

```
@PostMapping(value = "/produits" consumes = "application/json")
 public void ajouterUnProduit(@RequestBody Product produit) { … }
```

La méthode "*ajouterUnProduit*" sera invoquée suite à une requête HTTP qui utilise **la méthode HTTP "POST"** avec une URI qui pointe sur **la ressource "api/produits".**

Exemples

- **consumes** : spécifie le format des données d'entrées de la méthode (méthodes POST, PUT)
- **produces** : Spécifie le format de données renvoyées par la méthode (méthode GET)
- **Valeurs @Consumes / @Produces**
  - `application/json`
  - `application/xml`
  - `Text/plain`

Exemples

› **@RequestBody** : Permet à Spring de **convertir le contenu JSON ou XML envoyé dans la requête** vers une instance d'une classe Java.

```
@PostMapping(value = "/produits" consumes = "application/json")
public void ajouterUnProduit(@RequestBody Product produit) { … }
```

› Utilisé principalement lors d'un **POST** ou d'un **PUT**

Passage des paramètres

```
@GetMapping("/produits/{id}")
  public Product afficherUnProduit(@PathVariable("id") int id) { … }
```

- La méthode "afficherUnProduit" sera invoquée suite à une requête HTTP qui utilise **la méthode HTTP "GET"** avec une **URI dynamique** qui pointe sur **différentes ressources de Path variable « api/produits/{id}".**
- {id} est une variable utilisée dans le Path de l'URI.
- **@PathVariable** : utilisé lors des URLs dynamiques, pour récupérer la valeur de la variable utilisée dans le Path de l'URI dynamique.